AI 4 0 0 2 Computer Vision - Assignmen 3

Task: Rice Grain Counter

Roll No: 2 3 F- 0 0 6 0

Pipeline: load -> grayscale -> adaptive threshold -> noise inspection -> morphology -> separate t grains -> extract regions -> count -> compare -> analysis.

Step 1 : Load and visualize the image

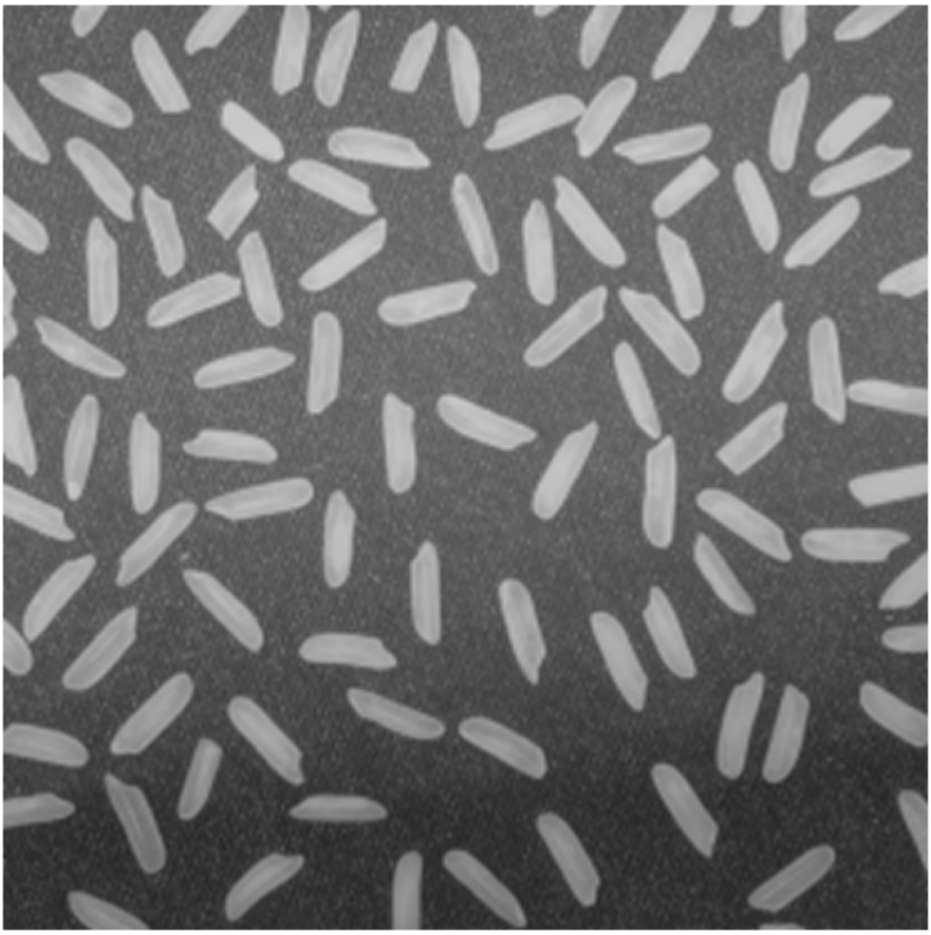
In [1]:
```
import cv2, numpy as np
import matplotlib.pyplot as plt

IMG_PATH = 'RiceGrains.png'   # keep the image in the same folder as this
img = cv2.imread(IMG_PATH)
assert img is not None, 'Image not found - check IMG_PATH'
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
print('Shape:', img.shape)

plt.figure(figsize=(6,6)); plt.imshow(img_rgb); plt.title('Original image
```

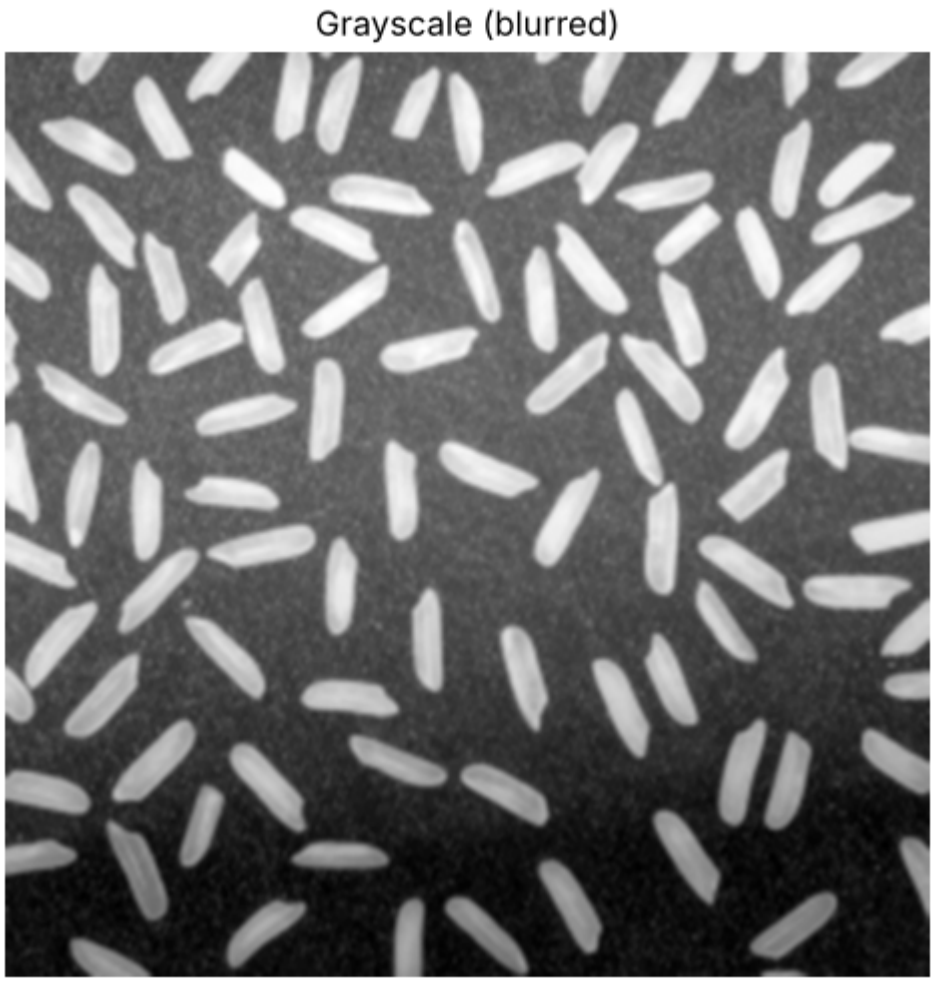
```
Shape: (512, 512, 3)
```

Original image



Step 2 : Convert to grayscale

In [2]:
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)   # light smoothing to suppress s
plt.figure(figsize=(6,6)); plt.imshow(blur, cmap='gray'); plt.title('Gray
```



Step 3 : Adaptive thresholding

The background is unevenly lit (darker in the corners), so one global threshold would fail. Adaptiv thresholding computes a local threshold per neighbourhood. Grains are brighter than the backgr `THRESH_BINARY` with a negative `C` is used.

In [3]:
```
binary = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, blockSize=51, C=-8)
plt.figure(figsize=(6,6)); plt.imshow(binary, cmap='gray'); plt.title('Ad
```



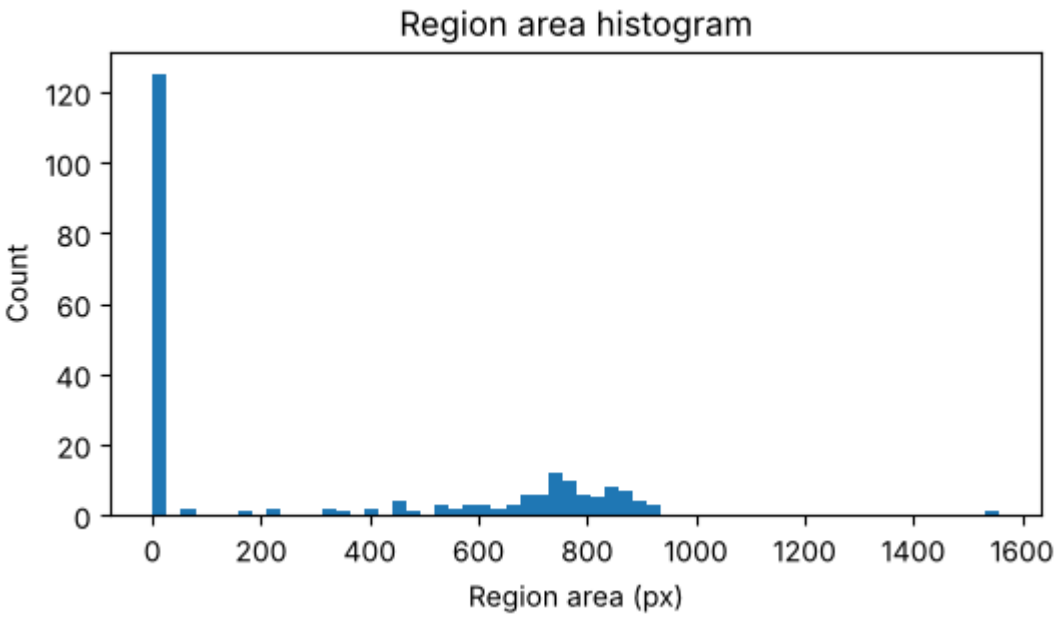
Step 4 : Identify unwanted small regions / noise

In [4]:
```
n, labels, stats, _ = cv2.connectedComponentsWithStats(binary, connectivi
areas = stats[1:, cv2.CC_STAT_AREA]
print('Components before cleaning:', len(areas))
print('Median area:', np.median(areas))

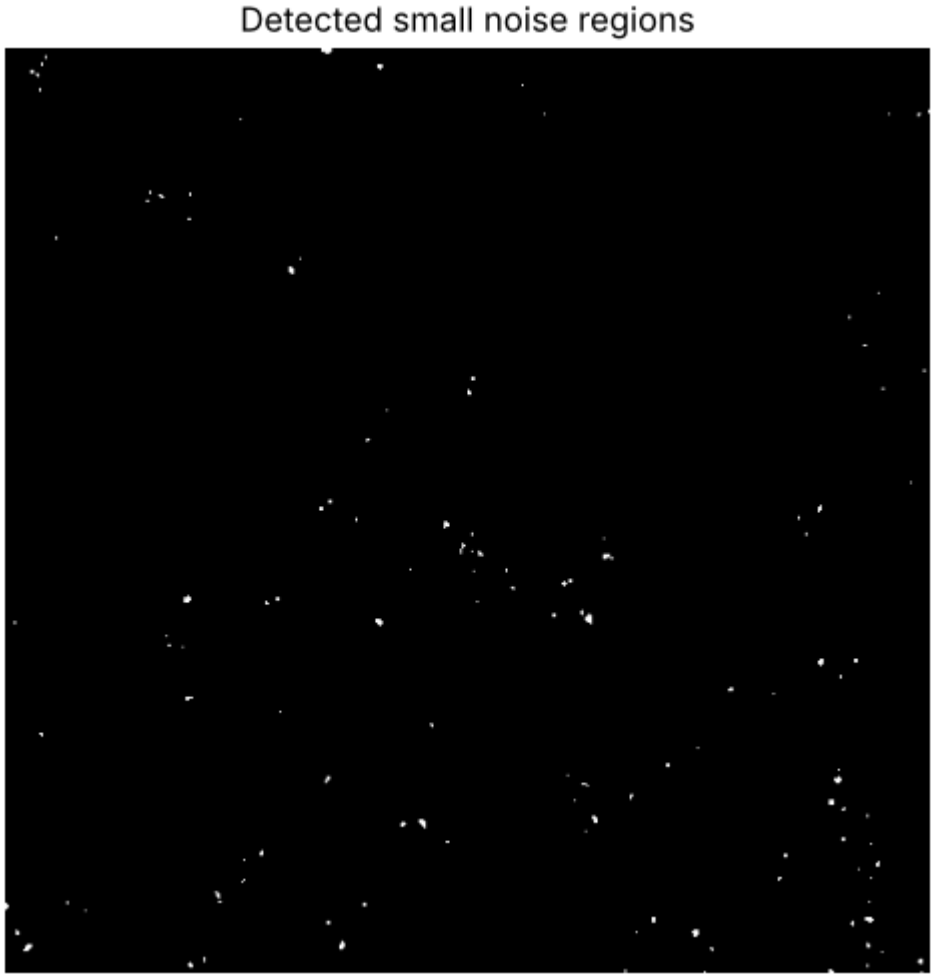
plt.figure(figsize=(6,3)); plt.hist(areas, bins=60); plt.xlabel('Region a
plt.title('Region area histogram'); plt.show()

NOISE_AREA = 30
noise_mask = np.zeros_like(binary)
for i in range(1, n):
    if stats[i, cv2.CC_STAT_AREA] < NOISE_AREA:
        noise_mask[labels == i] = 255
print('Tiny regions (< %d px):' % NOISE_AREA, int((areas < NOISE_AREA).su
plt.figure(figsize=(6,6)); plt.imshow(noise_mask, cmap='gray'); plt.title
```

```
Components before cleaning: 224
Median area: 9.0
```




```
Tiny regions (< 30 px): 125
```

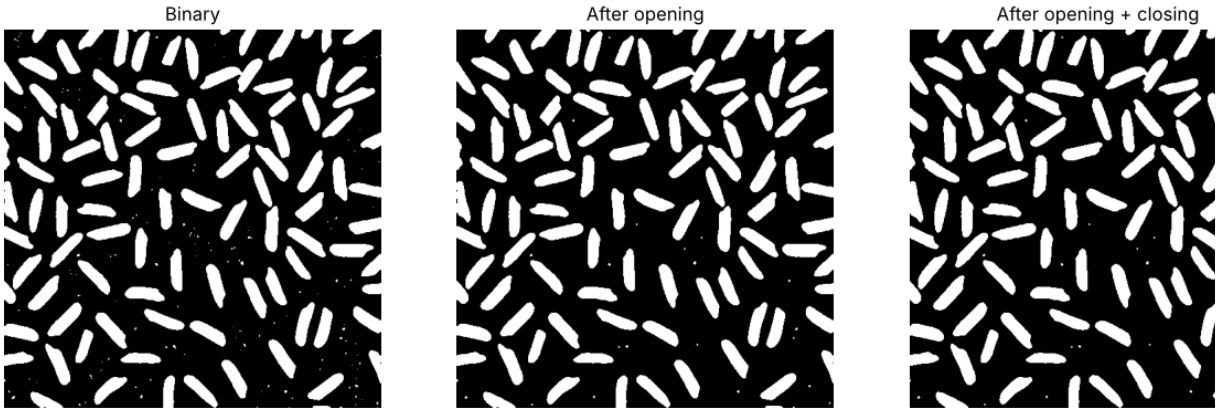


Step 5 : Morphological cleaning

Opening (erosion then dilation) is applied first because the noise is small *bright* specks on a dar background; opening removes objects smaller than the structuring element while keeping the gra shape. **Closing** with a small kernel then fills tiny holes inside grains caused by uneven brightnes

In [5]:
```
k3 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3,3))
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, k3)
cleaned = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, k3)

fig, ax = plt.subplots(1,3, figsize=(15,5))
for a, im, t in zip(ax, [binary, opened, cleaned], ['Binary', 'After open
    a.imshow(im, cmap='gray'); a.set_title(t); a.axis('off')
plt.show()
```



Step 6 : Effect of structuring-element size (and touch grains)

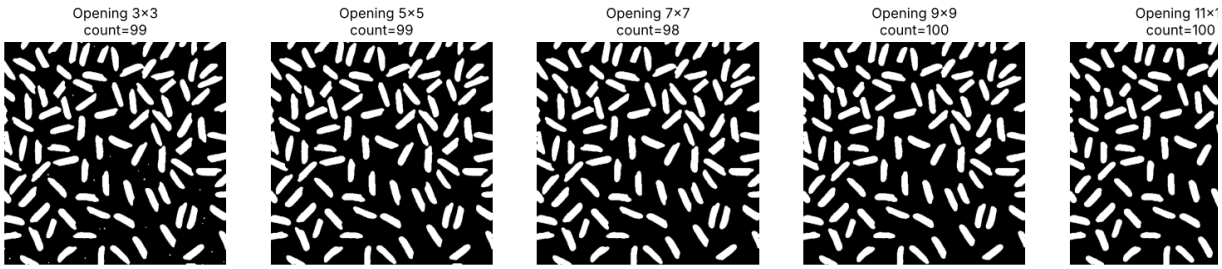
In [6]:
```
def count_components(mask, min_area=40):
    n, lab, st, _ = cv2.connectedComponentsWithStats(mask, connectivity=8
    return int((st[1:, cv2.CC_STAT_AREA] >= min_area).sum())

print('Count with no separation (cleaned image):', count_components(clean

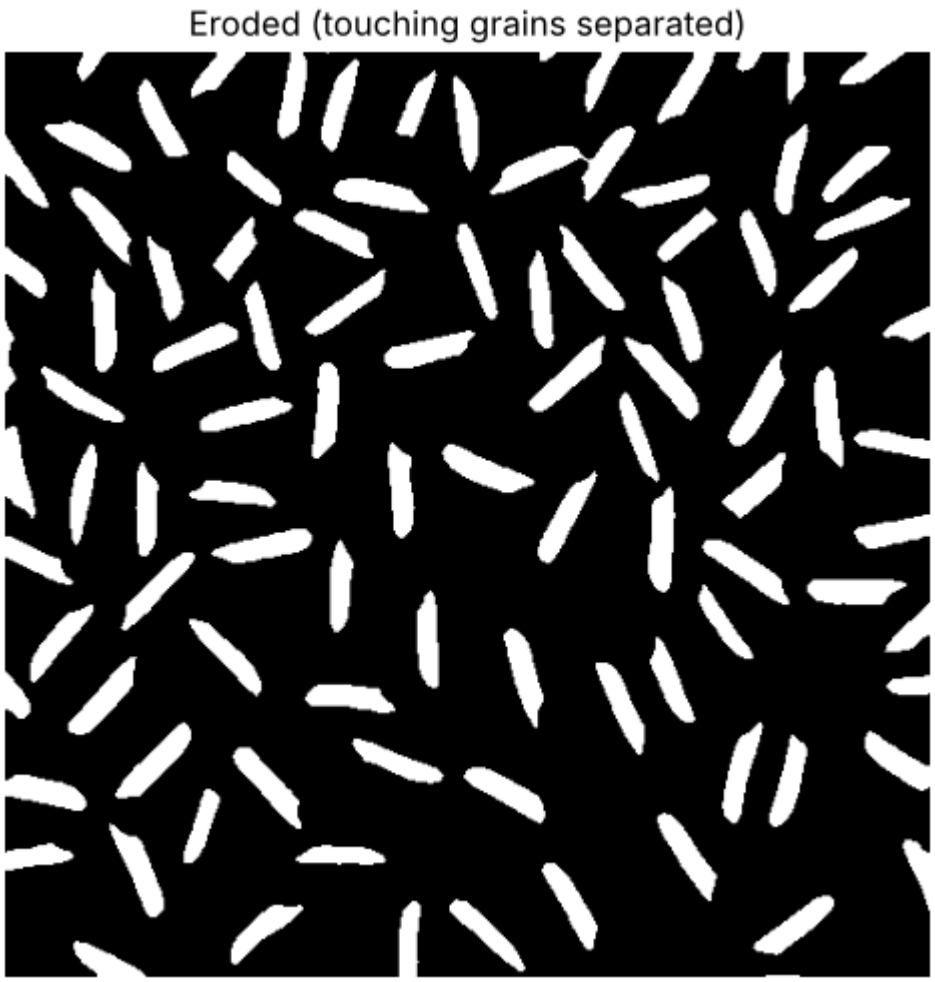
sizes = [3,5,7,9,11]
fig, ax = plt.subplots(1, len(sizes), figsize=(4*len(sizes), 4))
results = {}
for a, s in zip(ax, sizes):
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (s,s))
    op = cv2.morphologyEx(binary, cv2.MORPH_OPEN, k)
    results[s] = count_components(op)
    a.imshow(op, cmap='gray'); a.set_title(f'Opening {s}x{s}\ncount={resu
plt.show()
print('Opening size vs count:', results)

# Separation of touching grains: erosion splits thin necks between grains
ks = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
eroded = cv2.erode(cleaned, ks, iterations=1)
plt.figure(figsize=(6,6)); plt.imshow(eroded, cmap='gray'); plt.title('Er
```

```
Count with no separation (cleaned image): 98
```




```
Opening size vs count: {3: 99, 5: 99, 7: 98, 9: 100, 11: 100}
```



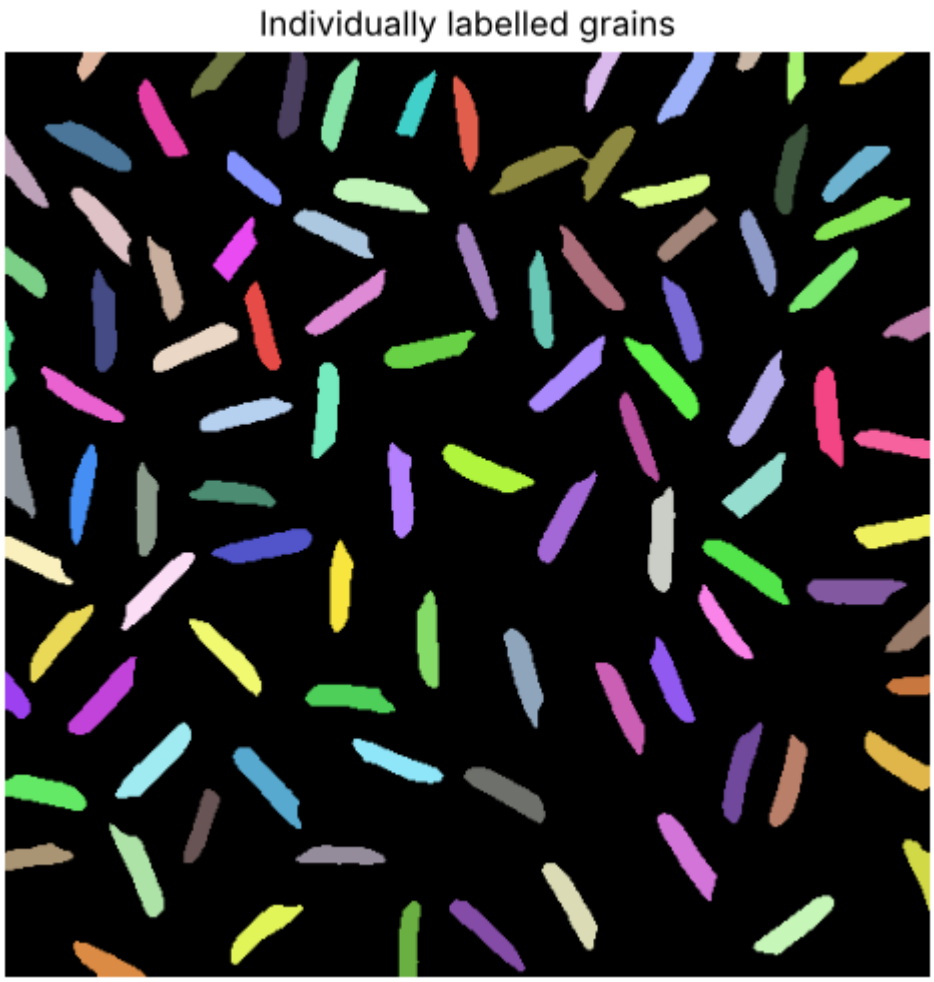
Step 7 : Extract individual grain regions

In [7]:
```
MIN_AREA = 40
n, labels, stats, cents = cv2.connectedComponentsWithStats(eroded, connec
grains = [i for i in range(1, n) if stats[i, cv2.CC_STAT_AREA] >= MIN_ARE
print('Extracted grain regions:', len(grains))

# Dilate each eroded region back (restricted to cleaned mask) to recover
extracted = np.zeros_like(cleaned)
for i in grains:
    extracted[labels == i] = 255
extracted = cv2.bitwise_and(cv2.dilate(extracted, ks, iterations=1), clea

colored = np.zeros((*gray.shape, 3), np.uint8)
rng = np.random.default_rng(0)
for i in grains:
    colored[labels == i] = rng.integers(60, 255, 3)
plt.figure(figsize=(6,6)); plt.imshow(colored); plt.title('Individually l
```

```
Extracted grain regions: 97
```



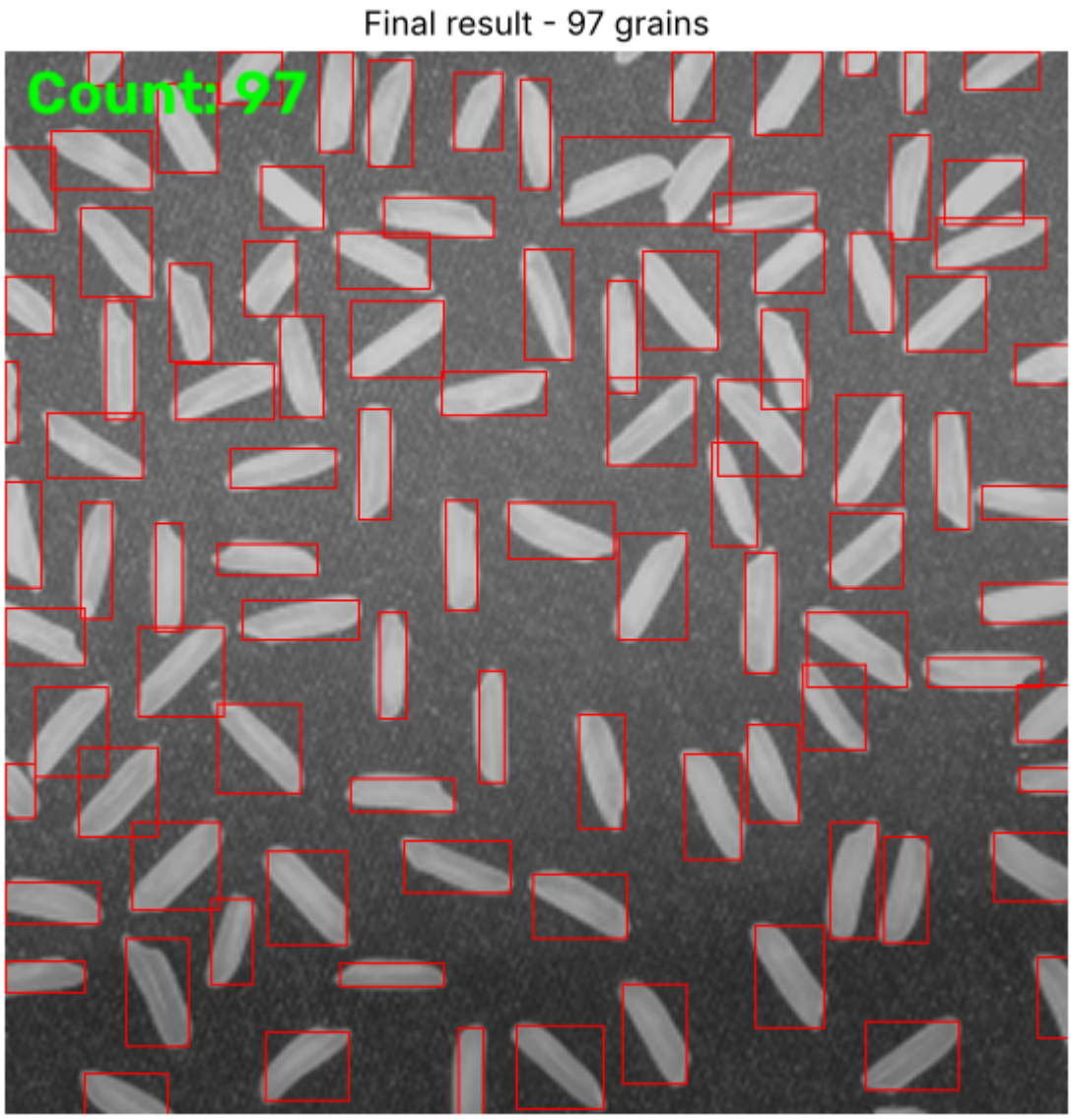
Step 8 : Count the grains

In [8]:
```
count = len(grains)
areas_g = stats[grains, cv2.CC_STAT_AREA]
print('Grain count:', count)
print('Mean area: %.1f  Median: %.1f' % (areas_g.mean(), np.median(areas_
```

```
Grain count: 97
Mean area: 479.6  Median: 499.0
```

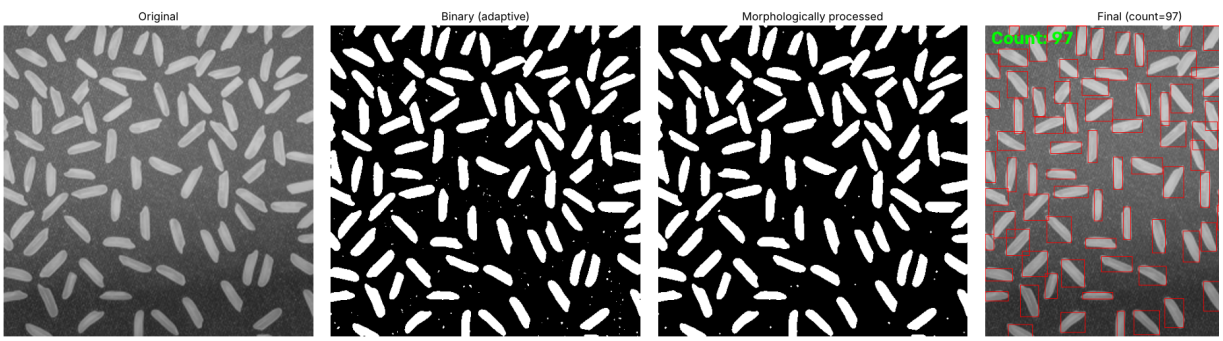
Step 9 : Final image with count

In [9]:
```
final = img_rgb.copy()
for i in grains:
    x, y, w, h = stats[i, :4]
    cv2.rectangle(final, (x, y), (x+w, y+h), (255, 0, 0), 1)
cv2.putText(final, f'Count: {count}', (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
plt.figure(figsize=(7,7)); plt.imshow(final); plt.title(f'Final result -
```



Step 10: Comparison

In [10]:
```
fig, ax = plt.subplots(1,4, figsize=(20,5))
for a, im, t, cm in zip(ax, [img_rgb, binary, cleaned, final],
                        ['Original', 'Binary (adaptive)', 'Morphologicall
                        [None, 'gray', 'gray', None]):
    a.imshow(im, cmap=cm); a.set_title(t); a.axis('off')
plt.tight_layout(); plt.show()
```



Step 11: Required analysis

(Results from this run: no separation = 98 grains; opening with 3x3 / 5x5 / 7x7 / 9x9 / 11x11 = 99 / 99 / 98 / 100 / final count after opening + closing + erosion = 97.)

1. **Why was Opening or Closing selected?** Opening was the main operation. After adaptive thresholding the noise is tiny isolated *white* specks (125 regions under 30 px, agai median grain area of about 500 px), and opening (erosion followed by dilation) deletes v objects smaller than the structuring element while restoring the size of the large grains. Closing v applied afterwards, only with a 3x3 element, to fill small black holes inside grains. Closing would not remove the specks and could merge neighbouring grains.

2. **What happened when the structuring-element size was increased?** With 3x3 mo single-pixel specks were removed; a few slightly larger specks survived and were only discarded area filter. From 5x5 upward the background is clean. At 9x9 and 11x11 clean-up is complete, but grain tips become rounder and shorter and thin grains start to erode, s elements risk damaging real grains. The count stayed in a narrow band (98 to 100) b the grains here are wide enough (roughly 10 to 15 px) to survive even 11x1

3. **Did morphology remove any actual rice grains?** Not in any meaningful way for sizes up t 11x11: the count stays at 98 to 100 and no whole grain visibly disappears size comparison figure. The final pipeline gave 97, which is 1 lower than the cleaned (98); this small drop comes from the erosion step, where tiny partial grains cut by the image b shrink below the 40 px minimum-area filter. So morphology can remove very small borde grains, but not full grains.

4. **How did touching grains affect the final count?** Touching grains merge into one connecte component and are counted once, which under-counts. The count rose from 98 (3x3 opening) to 100 (9x9 and 11x11 opening) because larger elements cut contacts between grains. The final erosion (5x5) did not give a higher count (97) because mild separation and also removes the border fragments mentioned above. A few grains that lie s side or cross each other along a long edge stay merged, so the true number is probably slightly l than 97.

5. **What limitation does the applied method have?**

- Grains that touch or overlap along a long edge cannot be separated by opening/erosion, so under-counted; very elongated grains could in principle also be over-split.
- Block size, C, structuring-element size and the minimum-area threshold are hand-tuned for image and lighting; other images may need different values.
- Partial grains on the image border are counted as whole grains (or dropped if too small).
- Distance-transform + watershed would separate touching grains better, but goes beyond the morphology taught in class.